Financial Risk Analysis with Python- Blackrock

Task 1: Data Cleaning and Formatting

- Importing of libraries and importing csv file

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[2]:

```
br=pd.read_csv(r"C:\Users\alpan\Downloads\blackrock.csv")
```

```
br.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 15 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   TransactionID      800 non-null    int64
 1   CustomerID         800 non-null    object
 2   AccountID          800 non-null    object
 3   AccountType        800 non-null    object
 4   TransactionType    800 non-null    object
 5   Product            800 non-null    object
 6   Firm               800 non-null    object
 7   Region             800 non-null    object
 8   Manager            800 non-null    object
 9   TransactionDate    800 non-null    object
 10  TransactionAmount  800 non-null    float64
 11  AccountBalance     800 non-null    float64
 12  RiskScore          800 non-null    float64
 13  CreditRating       800 non-null    int64
 14  TenureMonths       800 non-null    int64
dtypes: float64(3), int64(3), object(9)
memory usage: 93.9+ KB
```

Task 1: Data Cleaning and Formatting

Task 1.1: Remove/treat any special characters or non-numeric entries from financial fields.

```
categorical_columns= br.dtypes[br.dtypes=='object'].index
categorical_columns
```

```
Index(['CustomerID', 'AccountID', 'AccountType', 'TransactionType', 'Product',
       'Firm', 'Region', 'Manager', 'TransactionDate'],
      dtype='object')
```

```
numerical_columns= br.dtypes[(br.dtypes=='int64') | (br.dtypes=='float64')].index
numerical_columns
```

```
Index(['TransactionID', 'TransactionAmount', 'AccountBalance', 'RiskScore',
       'CreditRating', 'TenureMonths'],
      dtype='object')
```

- --> no finanacial columns containa any special charaters or non numerical values

Task 1.2: Convert currency amounts into numerical format.

```
br["TransactionAmount"]= pd.to_numeric(br["TransactionAmount"],errors='coerce')
```

```
br["AccountBalance"]= pd.to_numeric(br["AccountBalance"],errors='coerce')
```

```
br.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 15 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   TransactionID      800 non-null    int64
 1   CustomerID         800 non-null    object
 2   AccountID          800 non-null    object
 3   AccountType        800 non-null    object
 4   TransactionType    800 non-null    object
 5   Product            800 non-null    object
 6   Firm               800 non-null    object
 7   Region             800 non-null    object
 8   Manager            800 non-null    object
 9   TransactionDate    800 non-null    object
 10  TransactionAmount  800 non-null    float64
 11  AccountBalance     800 non-null    float64
 12  RiskScore          800 non-null    float64
 13  CreditRating       800 non-null    int64
 14  TenureMonths       800 non-null    int64
dtypes: float64(3), int64(3), object(9)
memory usage: 93.9+ KB
```

Task 1.3: Validate and format date columns.

```
br["TransactionDate"] = pd.to_datetime(br["TransactionDate"], dayfirst = True)
```

```
br.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 15 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   TransactionID      800 non-null    int64
 1   CustomerID         800 non-null    object
 2   AccountID          800 non-null    object
 3   AccountType        800 non-null    object
 4   TransactionType    800 non-null    object
 5   Product            800 non-null    object
 6   Firm               800 non-null    object
 7   Region             800 non-null    object
 8   Manager            800 non-null    object
 9   TransactionDate    800 non-null    datetime64[ns]
 10  TransactionAmount  800 non-null    float64
 11  AccountBalance     800 non-null    float64
 12  RiskScore          800 non-null    float64
 13  CreditRating       800 non-null    int64
 14  TenureMonths       800 non-null    int64
dtypes: datetime64[ns](1), float64(3), int64(3), object(8)
memory usage: 93.9+ KB
```

Task 1.4 Ensure account types and transaction categories are standardized. ¶

--> Account types and transaction categories are in standarized names.

```
br['AccountType'].unique().sum()
```

```
'CreditCurrentLoanSavings'
```

```
br['TransactionType'].unique().sum()
```

```
'WithdrawalPaymentDepositTransfer'
```

Task 2: Descriptive Transactional Analysis

Task 2.1: Calculate monthly and yearly summaries of total credits, debits, and net transaction volume. ¶

```
br['TransactionType'].value_counts()
```

```
TransactionType
Withdrawal    214
Transfer      208
Payment       196
Deposit       182
Name: count, dtype: int64
```

--> Total Credits = sum of Transaction Amount - Deposited (Transaction Type)

--> Total Debits = sum of Transaction amount of Withdrawal, Transfer and Payments done (Transaction Type)

```
br['Total_debits']=br['TransactionAmount'].where(br['TransactionType'].isin(['Withdrawal','Transfer','Payment']),0)
```

```
round(br['Total_debits'].sum(),2)
```

```
np.float64(32856407.47)
```

```
br['Total_credits']=br['TransactionAmount'].where(br['TransactionType']=='Deposit',0)
```

```
round(br['Total_credits'].sum(),2)
```

```
np.float64(10745085.9)
```

```
br['year']=br['TransactionDate'].dt.year
br['month']=br['TransactionDate'].dt.month
Transaction_summary = br.groupby(['year','month'])[['Total_credits','Total_debits']].sum()
Transaction_summary['Net_Transaction_Volume']=(Transaction_summary['Total_credits']-Transaction_summary['Total_debits'])
round(Transaction_summary)
```

year	month	Total_credits	Total_debits	Net_Transaction_Volume
2023	1	453288.0	1778891.0	-1325603.0
	2	556591.0	2610104.0	-2053512.0
	3	405511.0	1913691.0	-1508180.0
	4	328958.0	1770646.0	-1441688.0
	5	933559.0	2341659.0	-1408100.0
	6	676082.0	989464.0	-313382.0
	7	486550.0	1422898.0	-936348.0
	8	81985.0	1933709.0	-1851724.0
	9	749147.0	2195770.0	-1446623.0
	10	1322052.0	2846267.0	-1524216.0
	11	966178.0	1984515.0	-1018337.0
	12	522862.0	1204102.0	-681240.0
2024	1	792453.0	1109510.0	-317057.0
	2	624446.0	2076592.0	-1452146.0
	3	765295.0	2623592.0	-1858297.0
	4	376351.0	1294163.0	-917812.0
	5	300787.0	1335151.0	-1034364.0
	6	402990.0	1425684.0	-1022694.0

Task: 2.2 Plot Trends in total credits vs debits over time

--> balancechange is the difference Account balance occurs due to credit and debit in accounts

--> net_credit is the sum of balance where the account balance is more than 0.

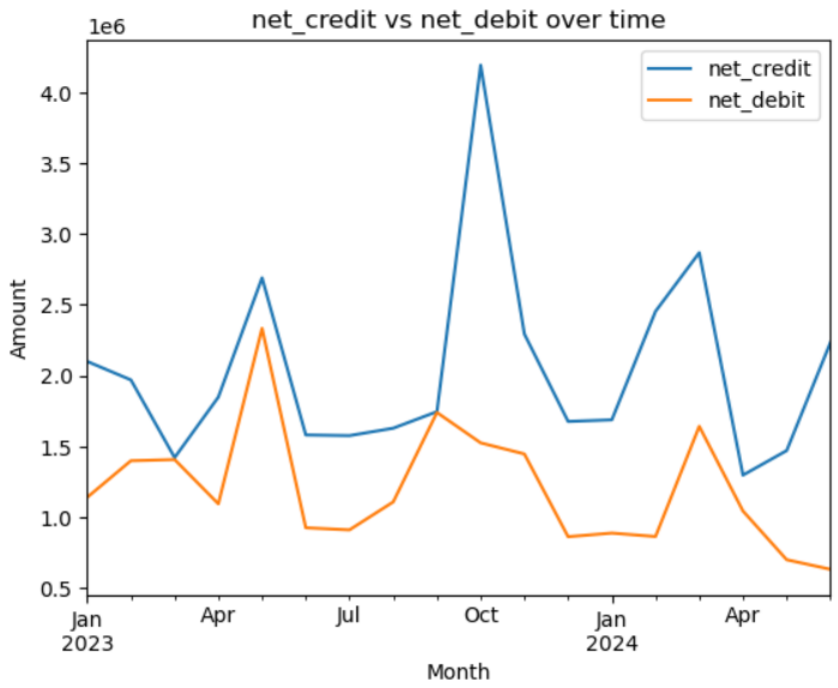
--> net_debit is the sum of balance where the account balance is less than 0.

--> we plot the trend of total credit and debit over months not years ¶

```
br['balancechange']= br['AccountBalance'].diff()
br['net_credit'] = br['AccountBalance'].where (br['balancechange']>0,0)
br['net_debit'] = br['AccountBalance'].where (br['balancechange']<0,0)
```

```
monthly_trend= br.groupby(br['TransactionDate'].dt.to_period('M'))[['net_credit','net_debit']].sum()
```

```
import matplotlib.pyplot as plt
monthly_trend.plot()
plt.title("net_credit vs net_debit over time")
plt.xlabel('Month')
plt.ylabel('Amount')
plt.show()
```



Task: 2.3 Identify top and bottom performing accounts based on net flow

```
Unique_account_detail= br.groupby('AccountID')[['net_credit','net_debit']].sum()
Unique_account_detail['net_flow'] = Unique_account_detail['net_credit']-Unique_account_detail['net_debit']
```

5 Top Performing Accounts based on net flow

```
top_performing_accounts = Unique_account_detail.sort_values(by='net_flow', ascending=False).head(5)
```

```
top_performing_accounts
```

AccountID	net_credit	net_debit	net_flow
ACC29231	612538.30982	0.00000	612538.30982
ACC28295	500173.23655	0.00000	500173.23655
ACC29646	507645.86480	23430.49327	484215.37153
ACC57700	510872.99761	69920.35629	440952.64132
ACC64022	387964.27030	-11356.97561	399321.24591

5 Bottom Performing Accounts based on net flow ¶

```
bottom_performing_accounts = Unique_account_detail.sort_values(by='net_flow', ascending=True).head(5)
```

```
bottom_performing_accounts
```

AccountID	net_credit	net_debit	net_flow
ACC43771	80044.22591	393897.84694	-313853.62103
ACC78178	0.00000	292569.07038	-292569.07038
ACC95774	115204.97170	375083.88241	-259878.91071
ACC42467	0.00000	211164.60916	-211164.60916
ACC42903	0.00000	209012.95915	-209012.95915

Task 2.4: Identify and flag accounts as dormant or inactive if there is a gap of two months or more between consecutive transactions.

--> prev_transaction is the column which tells the previous transaction date of the same account ID thus we need to first sort the AccountID and then TransactionDate to get the correct difference of days between two transaction by the same account

--> gap_days calculates the total days between two transactions by the same account

--> account_status_flag is the column which flags the accounts which have

--> gap of more than 60 days between two transactions as Inactive

--> accounts with less than 60 gap days are Active

```
br=br.sort_values(['AccountID','TransactionDate'])
br['prev_transaction']= br.groupby('AccountID')['TransactionDate'].shift(1)
br['gap_days']=(br['TransactionDate']-br['prev_transaction']).dt.days
br['max_gap']= br.groupby('AccountID')['gap_days'].transform('max')
br['account_status']=br['max_gap'].apply(lambda x: 'Inactive' if x>=60 else'Active')
account_status_flag = br[['AccountID', 'account_status']].drop_duplicates()
account_status_flag['account_status'].value_counts()
```

```
account_status
Inactive    172
Active       20
Name: count, dtype: int64
```

--> Out of 192 unique accounts 172 accounts are inactive and 20 accounts are active.

Task 3: Customer Profile Building

Task 3.1: Group Accounts by activity levels: High, Medium, Low based on transaction frequency on your analysis and rubrics. Do not forget to mention the rubric in the headings.

```
br['Total_Transactions'] = br.groupby('CustomerID')['TransactionID'].transform('count')
br['Total_Transactions']
```

```
549    3
301    5
724    5
787    2
27     6
      ..
398    5
336    4
303    6
139    6
581    7
Name: Total_Transactions, Length: 800, dtype: int64
```

--> We define a function to segment the Customers by activity level

--> If Total_Transactions >10 then Activity level marked as High

--> If Total_Transactions =>5 then Activity level marked as Medium

--> If Total_Transactions <5 then Activity level marked as Low

```
def activity_level(x):
    if x > 10:
        return"High"
    elif x>=5:
        return"Medium"
    else:
        return "Low"
br['activity_level'] =br['Total_Transactions'].apply(activity_level)
activity_level
```

```
<function __main__.activity_level(x)>
```

```
br[['CustomerID','activity_level']].drop_duplicates()['activity_level'].value_counts()
```

```
activity_level
Low       97
Medium    82
High       1
Name: count, dtype: int64
```

--> Customer activity segementation : Customers were segmented into activity levels based on the number of transactions performed

Task 3.2: Segment customers by average balance and transaction volume. ¶

```
avg_balance= br.groupby('CustomerID')['AccountBalance'].mean().reset_index(name='Avg_Balance')
transaction_volume = br.groupby('CustomerID')['TransactionID'].count().reset_index(name='Transaction_count')
customer_details=avg_balance.merge(transaction_volume, on = 'CustomerID')
```

Define a function to segment the customers based on different conditions

--> If Average Balance is more than 20000 and Transaction count is more than 8 mark as Premium customer

--> If Average Balance is >= 20000 and Transaction count is >= 4 mark as Elite customer

--> If Average Balance is >= 5000 and Transaction count is >=2 mark as Basic customer

--> Else mark as New customer

```
def segment(data):
    if data['Avg_Balance'] >20000 and data['Transaction_count']>8:
        return"Premium"
    elif data['Avg_Balance'] >= 20000  and data['Transaction_count']>=4:
        return"Elite"
    elif data['Avg_Balance'] >= 5000 and data['Transaction_count']>=2:
        return"Basic"
    else:
        return "New"
customer_details['segment_customers']= customer_details.apply(segment , axis = 1)
```

```
customer_details.head(10)
```

	CustomerID	Avg_Balance	Transaction_count	segment_customers
0	CUST1042	79417.266317	6	Elite
1	CUST1114	51474.223220	2	Basic
2	CUST1121	63924.640954	5	Elite
3	CUST1189	39051.942297	3	Basic
4	CUST1223	76370.221527	7	Elite
5	CUST1376	80779.647074	5	Elite
6	CUST1467	56689.613536	5	Elite
7	CUST1497	90564.708385	4	Elite
8	CUST1498	65424.298341	8	Elite
9	CUST1547	92293.667688	6	Elite

```
customer_details['segment_customers'].value_counts()
```

```
segment_customers
Elite      109
Basic       56
New          8
Premium      7
Name: count, dtype: int64
```

Task 3.3: Create profiles for:

Task 3.3.1: High-net inflow accounts

To get the top 5 High inflow accounts we find calculate the overall_net_inflow by AccountID

then sort the values in descending order and extract only top 5 accounts with high inflow

```
overall_net_flow = br.groupby('AccountID')['AccountBalance'].sum().reset_index(name='net_flow')
net_inflow_accounts= overall_net_flow.sort_values(by='net_flow',ascending=False).head(5)
net_inflow_accounts
```

	AccountID	net_flow
77	ACC41829	801172.534718
13	ACC16241	764175.627330
3	ACC11188	722116.360500
188	ACC97411	715262.630650
75	ACC40939	659794.704480

```
net_inflow_accounts.describe()
```

	net_flow
count	5.000000
mean	732504.371536
std	53409.430730
min	659794.704480
25%	715262.630650
50%	722116.360500
75%	764175.627330
max	801172.534718

As we see minimum net_inflow_accounts value is 659794 and maximum value is 801172 and mean is 732504

--> Then we set a criteria to know the Accounts whose inflow is more than 700000

```
high_net_inflow_accounts =overall_net_flow[overall_net_flow['net_flow']>700000]
high_net_inflow_accounts
```

	AccountID	net_flow
3	ACC11188	722116.360500
13	ACC16241	764175.627330
77	ACC41829	801172.534718
188	ACC97411	715262.630650

--> So there are only 4 out of 192 accounts with high inflow value of more than 700000.

Task 3.3.2: High-frequency low-balance accounts

--> We consider Account have High-Frequency when Total_Transactions is more than 8

--> We consider Account have Low balance when AccountBalance is less than 50000

```
high_frq_low_bal_accounts = br[(br['Total_Transactions'] >= 8) & (br['AccountBalance'] < 50000)]
customer_profile_2=high_frq_low_bal_accounts[['AccountID','AccountBalance','Total_Transactions']]
customer_profile_2
```

157	ACC49140	37584.564060	11
345	ACC55729	46054.339790	8
713	ACC57516	49558.858580	9
380	ACC57700	22188.806360	8
740	ACC61827	49298.638660	9
435	ACC71938	23713.158170	9
617	ACC75675	243.761772	9
64	ACC78581	29070.041550	8
107	ACC78589	-25971.512010	9
638	ACC82298	46317.100170	9
508	ACC96277	25255.512870	9
372	ACC96277	49672.888520	9

--> 16 out of 192 accounts are High-frequency low- Balance accounts

Task 3.3.3: Accounts with negative or near-zero balances

```
br['AccountBalance'].median()
```

```
73006.75195
```

--> We considered Accounts which have Balance <= 10 are near_zero_bal_accounts

```
near_zero_bal_accounts= br[br['AccountBalance']<= 10]
customer_profile_3 =near_zero_bal_accounts[['AccountID', 'AccountBalance']]
customer_profile_3.describe()
customer_profile_3
```

	AccountID	AccountBalance
7	ACC11062	-9084.224009
511	ACC15671	-8225.602340
442	ACC22036	-4547.371459
402	ACC30787	-3069.411127
642	ACC34821	-32532.523740
733	ACC39529	-35891.001750
389	ACC45101	-505.916206
739	ACC45521	-18963.797190
258	ACC46953	-26968.230900
432	ACC64022	-11356.975610
115	ACC74631	-8820.480676
107	ACC78589	-25971.512010
488	ACC78589	-4503.284142
251	ACC87602	-5518.347956
137	ACC88252	-7644.952475
172	ACC88516	-9208.204563
678	ACC94907	-7738.893700
196	ACC97225	-3049.407252

--> 18 out of 192 accounts have near or zero balance

Task 4: Financial Risk Identification

Task 4.1: Track accounts with frequent large withdrawals or overdrafts.

```
Q1 = br['TransactionAmount'].quantile(0.25)
Q3 = br['TransactionAmount'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
overdrafts = br[(br['TransactionAmount'] < lower_bound) |
              (br['TransactionAmount'] > upper_bound)]
overdrafts[['AccountID','TransactionAmount']]
overdrafts
```

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager	TransactionDate	...	year	month	balancechange	ne
264	133	CUST4780	ACC18057	Savings	Payment	Personal Loan	Firm D	Central	Manager 2	2023-11-25	...	2023	11	-5175.82673	
168	141	CUST2443	ACC21719	Current	Deposit	Home Loan	Firm E	North	Manager 2	2023-06-18	...	2023	6	43476.24652	6734
306	190	CUST6565	ACC41829	Loan	Payment	Home Loan	Firm B	Central	Manager 3	2024-03-12	...	2024	3	-10071.45836	
755	30	CUST1888	ACC45101	Credit	Payment	Savings Account	Firm A	Central	Manager 1	2023-04-19	...	2023	4	70719.74755	11034
377	189	CUST1042	ACC83848	Loan	Payment	Personal Loan	Firm B	West	Manager 1	2024-02-09	...	2024	2	-60456.72974	
481	20	CUST7793	ACC88449	Current	Transfer	Credit Card	Firm D	North	Manager 4	2023-06-19	...	2023	6	-39727.78812	
640	67	CUST1962	ACC94203	Savings	Deposit	Home Loan	Firm C	South	Manager 2	2024-06-07	...	2024	6	54786.72209	1014(

7 rows × 28 columns

--> 7 out of 192 Accounts are having frequent large withdrawals or overdrafts.

Task 4.2: Calculate balance volatility using standard deviation or coefficient of variation.

```
br['AccountBalance'].describe()
```

```
count       800.000000
mean      72947.111696
std       34206.072059
min      -35891.001750
25%       51133.057895
50%       73006.751950
75%       94553.225110
max      171767.657800
Name: AccountBalance, dtype: float64
```

```
balance_volatility = br.groupby('AccountID')['AccountBalance'].std().reset_index(name='Balance_volatility')
balance_volatility['Balance_volatility'] =balance_volatility['Balance_volatility'].fillna(0)
round(balance_volatility,2).sort_values(by='Balance_volatility', ascending= False)
```

	AccountID	Balance_volatility
73	ACC39529	91165.11
89	ACC46953	88564.26
154	ACC78589	86423.15
183	ACC95164	70375.13
21	ACC20297	70164.03
...	...	...
100	ACC50817	0.00
127	ACC65545	0.00
151	ACC78089	0.00
147	ACC77533	0.00
189	ACC99117	0.00

192 rows × 2 columns

--> Balance volatility measure the fluctuation in account balance over time.

--> It was calculated using the standard deviation of account balances for each account. ¶

--> Accounts with only one transaction resulted in 0, assuming no variability in balance.

Task 4.3: Use IQR or z-score methods to detect anomalies.

```
Q1= br['AccountBalance'].quantile(0.25)
Q3= br['AccountBalance'].quantile(0.75)
IQR=Q3-Q1
lower_bound= Q1- 1.5*IQR
upper_bound= Q3+ 1.5*IQR
outliers = br[(br['AccountBalance']<lower_bound) |(br['AccountBalance']>upper_bound)]
outliers[['AccountID','AccountBalance']]
outliers
```

	TransactionID	CustomerID	AccountID	AccountType	TransactionType	Product	Firm	Region	Manager
444	15	CUST4769	ACC28295	Current	Withdrawal	Home Loan	Firm B	East	Manager 4
117	97	CUST4780	ACC29646	Savings	Deposit	Savings Account	Firm D	North	Manager 4
642	170	CUST3315	ACC34821	Savings	Transfer	Credit Card	Firm E	West	Manager 3
733	65	CUST6565	ACC39529	Savings	Withdrawal	Credit Card	Firm B	East	Manager 2
739	102	CUST1962	ACC45521	Savings	Withdrawal	Home Loan	Firm C	South	Manager 4
258	68	CUST9731	ACC46953	Loan	Payment	Credit Card	Firm A	West	Manager 3
352	101	CUST4346	ACC65545	Current	Payment	Mutual Fund	Firm D	North	Manager 2
559	161	CUST4684	ACC66086	Credit	Transfer	Savings Account	Firm C	South	Manager 1
349	88	CUST5705	ACC71426	Current	Deposit	Credit Card	Firm B	North	Manager 4
309	135	CUST6735	ACC76549	Loan	Payment	Mutual Fund	Firm B	East	Manager 2
107	54	CUST2578	ACC78589	Credit	Withdrawal	Savings Account	Firm A	South	Manager 4
437	148	CUST7345	ACC88252	Credit	Deposit	Home Loan	Firm B	West	Manager 2

--> 12 Out of 192 Accounts have anomalies they are outliers, which means that some balance values for an account are unusually high or unusually low compared to the typical balances of that account.

Task 4.4: Highlight customers with irregular or suspicious transaction behavior.

--> Customers which are having transaction amount difference more than 100000

```
br['prev_transaction_amount'] = br.groupby('CustomerID')['TransactionAmount'].shift(1)
br['Amount_change'] = br['TransactionAmount']-br['prev_transaction_amount']
suspicious_customers = br[br['Amount_change'] >100000]
suspicious_customers[['CustomerID','Amount_change']]
```

	CustomerID	Amount_change
306	CUST6565	104273.459670
344	CUST3041	126352.703240
277	CUST2612	106284.097400
578	CUST4438	127626.065880
435	CUST4684	102359.625056
22	CUST5500	107008.076827
377	CUST1042	144958.270128
53	CUST7793	107160.695740
632	CUST1962	124689.478770
689	CUST2412	105561.566280

--> 10 out of 180 Customers have suspecious change in bwtween two consecutive transaction amount.

--> Customers having gap days more than 365 irregular customers

```
irregular_customers = br[br['gap_days'] >365]
irregular_customers[['CustomerID','gap_days']]
```

	CustomerID	gap_days
436	CUST7933	380.0
702	CUST4780	368.0
362	CUST5121	394.0
246	CUST2871	435.0
211	CUST5253	403.0
424	CUST5916	389.0
113	CUST1888	378.0
76	CUST2443	387.0

--> 8 out of 180 Customers have gap days between two transactions more than 365 that is a year.

Task 5: Visualisation

```
br.shape
```

```
(800, 30)
```

```
br.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 800 entries, 549 to 581
Data columns (total 30 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   TransactionID            800 non-null    int64
 1   CustomerID               800 non-null    object
 2   AccountID                800 non-null    object
 3   AccountType              800 non-null    object
 4   TransactionType          800 non-null    object
 5   Product                  800 non-null    object
 6   Firm                     800 non-null    object
 7   Region                   800 non-null    object
 8   Manager                  800 non-null    object
 9   TransactionDate          800 non-null    datetime64[ns]
 10  TransactionAmount        800 non-null    float64
 11  AccountBalance           800 non-null    float64
 12  RiskScore                800 non-null    float64
 13  CreditRating             800 non-null    int64
 14  TenureMonths             800 non-null    int64
 15  debit                    800 non-null    float64
 16  Total_credits            800 non-null    float64
 17  Total_debits             800 non-null    float64
 18  year                     800 non-null    int32
 19  month                    800 non-null    int32
 20  balancechange            799 non-null    float64
 21  net_credit               800 non-null    float64
 22  net_debit                800 non-null    float64
 23  prev_transaction         608 non-null    datetime64[ns]
 24  gap_days                 608 non-null    float64
 25  status                   800 non-null    object
 26  Total_Transactions       800 non-null    int64
 27  activity_level           800 non-null    object
 28  prev_transaction_amount  620 non-null    float64
 29  Amount_change            620 non-null    float64
dtypes: datetime64[ns](2), float64(12), int32(2), int64(4), object(10)
memory usage: 187.5+ KB
```

```
categorical_columns= br.dtypes[br.dtypes=='object'].index
categorical_columns
```

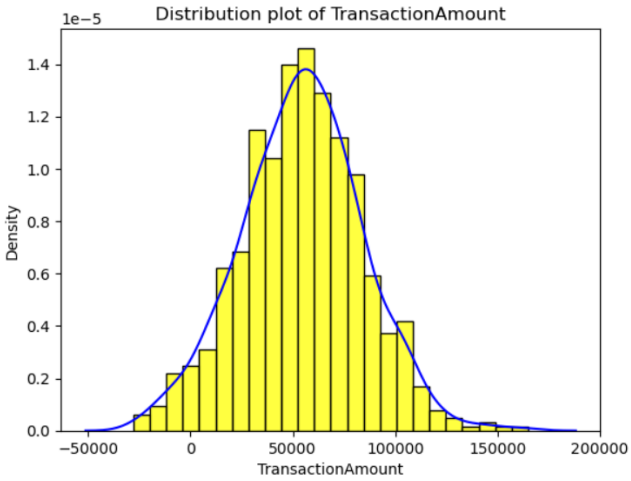
```
Index(['CustomerID', 'AccountID', 'AccountType', 'TransactionType', 'Product',
       'Firm', 'Region', 'Manager', 'status', 'activity_level'],
      dtype='object')
```

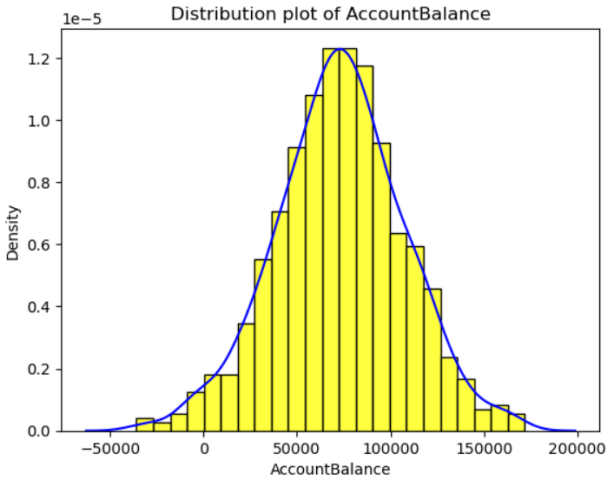
```
numerical_columns= br.dtypes[(br.dtypes=='int64') | (br.dtypes=='float64')].index
numerical_columns
```

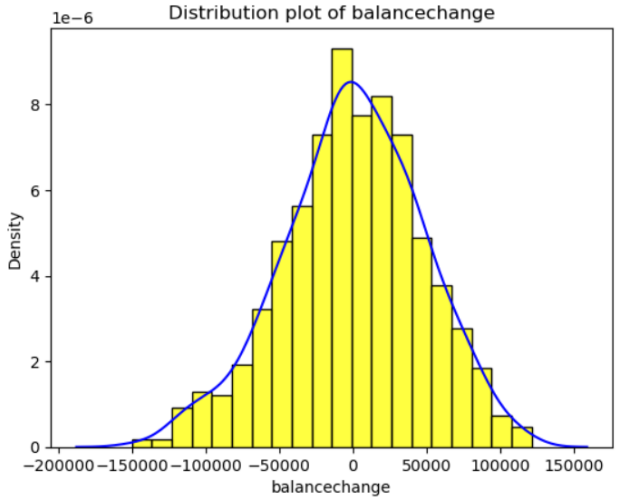
```
Index(['TransactionID', 'TransactionAmount', 'AccountBalance', 'RiskScore',
       'CreditRating', 'TenureMonths', 'debit', 'Total_credits',
       'Total_debits', 'balancechange', 'net_credit', 'net_debit', 'gap_days',
       'Total_Transactions', 'prev_transaction_amount', 'Amount_change'],
      dtype='object')
```

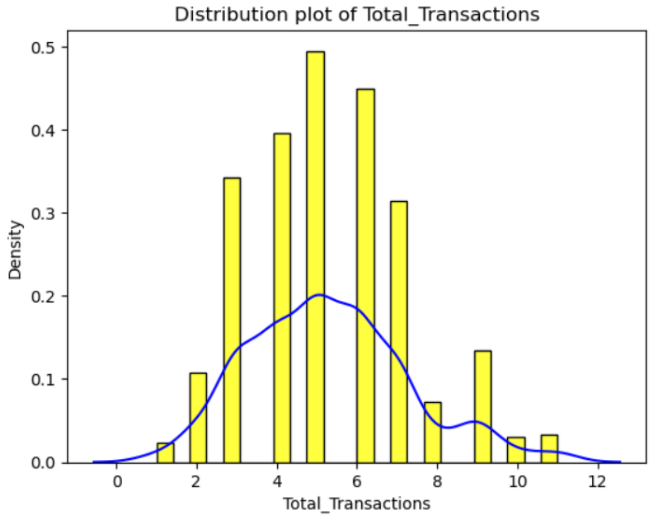
```
for col in numerical_columns:
    sns.histplot(br[col], stat= "density", color ="yellow")
    sns.kdeplot(br[col], color= "Blue", linewidth=1.5)
    plt.title(f"Distribution plot of {col}")
    plt.show()
```

Various graphs produced for this dataset are:

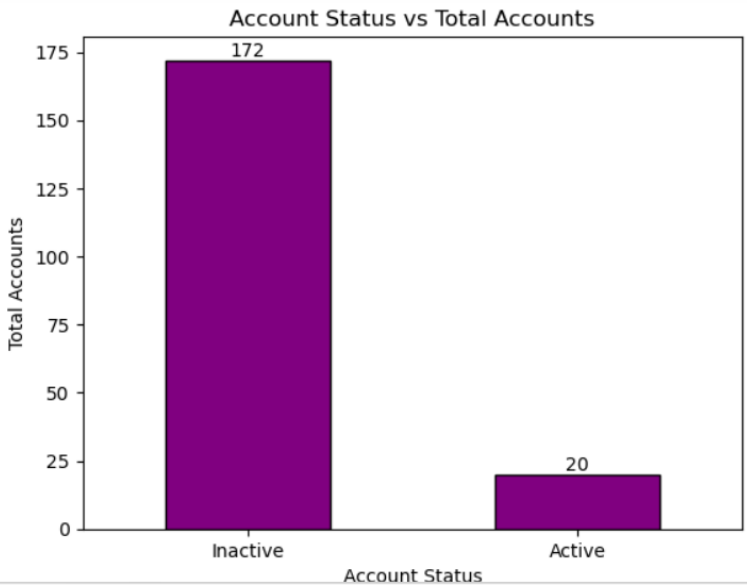






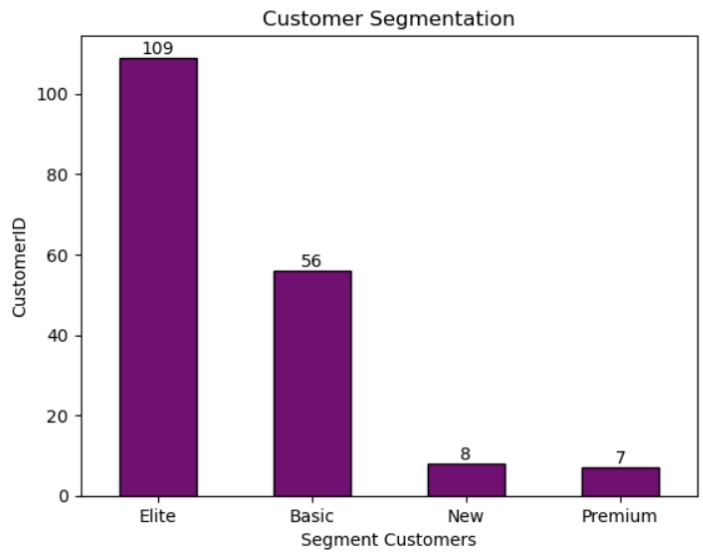


```
asplot=account_status_flag['account_status'].value_counts().plot(kind='bar',color='purple',edgecolor='black',width=0.5)
for container in asplot.containers:
    asplot.bar_label(container)
plt.title("Account Status vs Total Accounts")
plt.xlabel("Account Status")
plt.ylabel("Total Accounts")
plt.xticks(rotation=0)
plt.show()
```



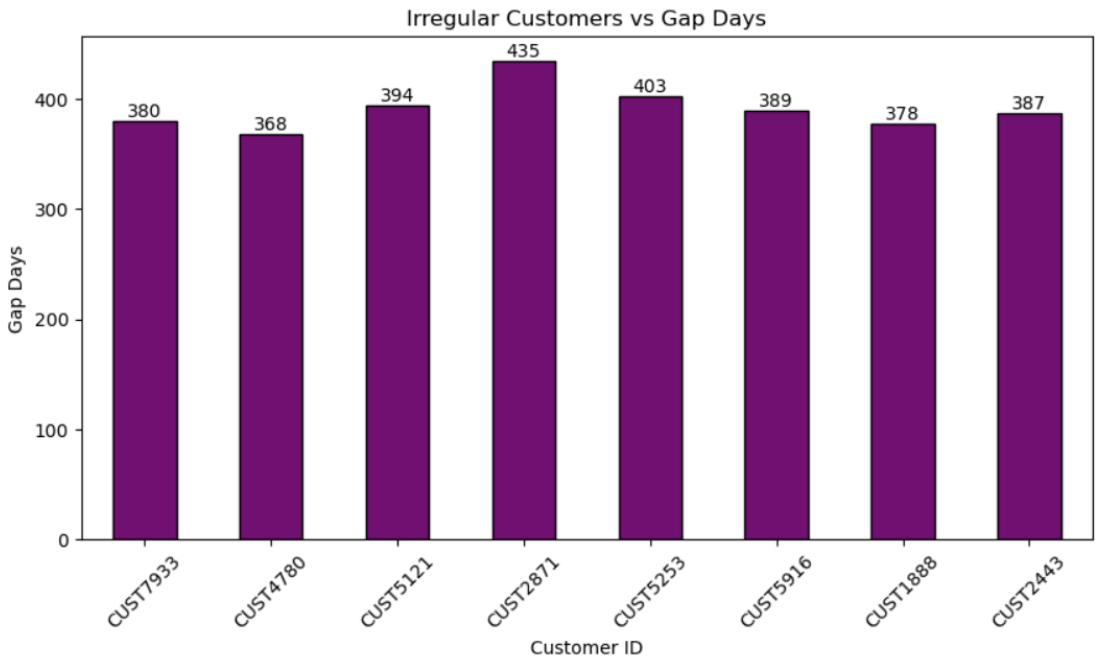
There are 172 out of 192 accounts that is 89.5 % accounts are Inactive rest 20 which is 10.5% of 192 are Active Accounts.

```
splot = sns.countplot(x='segment_customers', data=customer_details,color='purple',edgecolor='black',width=0.5)
for container in splot.containers:
    splot.bar_label(container)
plt.title("Customer Segmentation")
plt.xlabel("Segment Customers")
plt.ylabel("CustomerID")
plt.show()
```



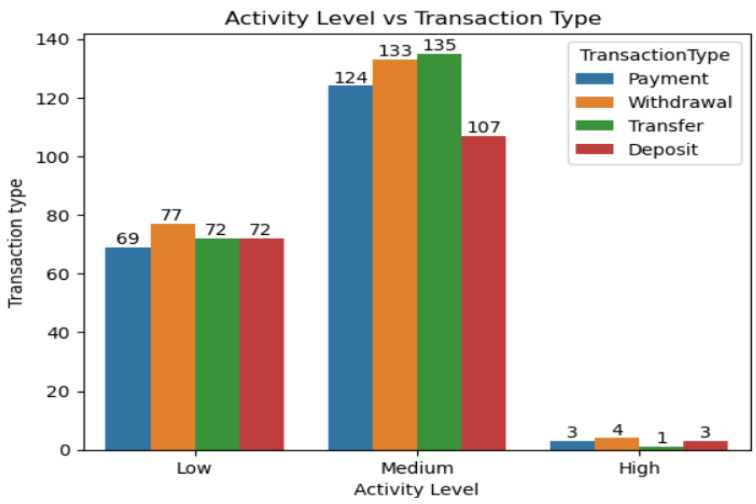
Out of 180 customers, 109 are Elite (high balance and moderate transactions), 56 are basic (low balance and low transactions), 8 are new customers with 1 transaction and the rest are premium customers with high balance and high transactions.

```
plt.figure(figsize=(10,5))
gdplot = sns.barplot(x='CustomerID', y='gap_days', data=irregular_customers,color='purple',edgecolor='black',width=0.5)
for container in gdplot.containers:
    gdplot.bar_label(container)
plt.xticks(rotation=45)
plt.title("Irregular Customers vs Gap Days")
plt.xlabel("Customer ID")
plt.ylabel("Gap Days")
plt.show()
```



8 out of 180 Customers have gap days between two transactions more than 365 days maximum gap day between two transaction is 435 by customerID CUST2871

```
atplot=sns.countplot(x=br['activity_level'],hue= br['TransactionType'])
for container in atplot.containers:
    atplot.bar_label(container)
plt.title('Activity Level vs Transaction Type ')
plt.xlabel( 'Activity Level')
plt.ylabel( 'Transaction type')
plt.xticks(rotation=0)
plt.show()
```



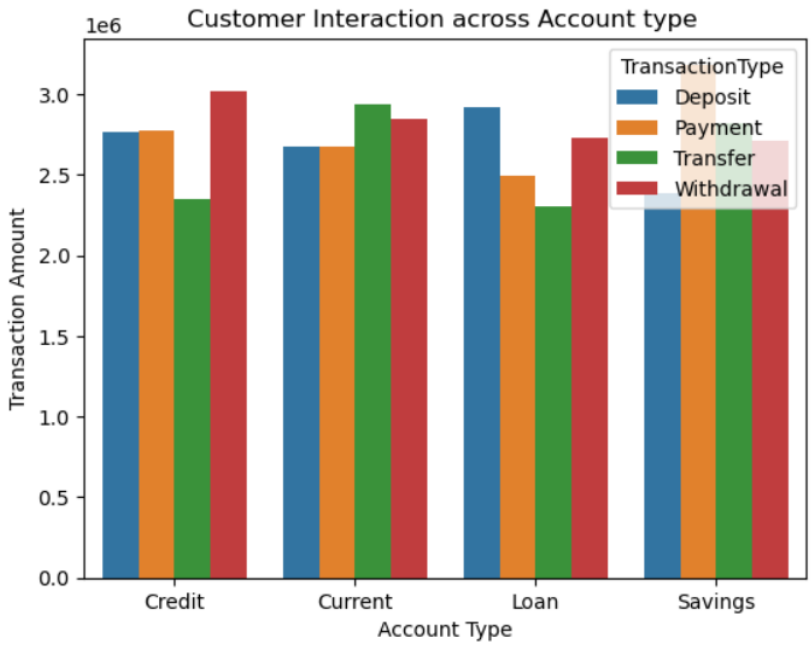
Medium activity customers are the most engaged and contribute the majority of transactions.

Low activity users still perform all types of transactions but at a lower frequency.

Very few customers fall into the “high activity” category.

Transaction behavior is fairly balanced across types, with slightly higher preference for transfers and withdrawals.

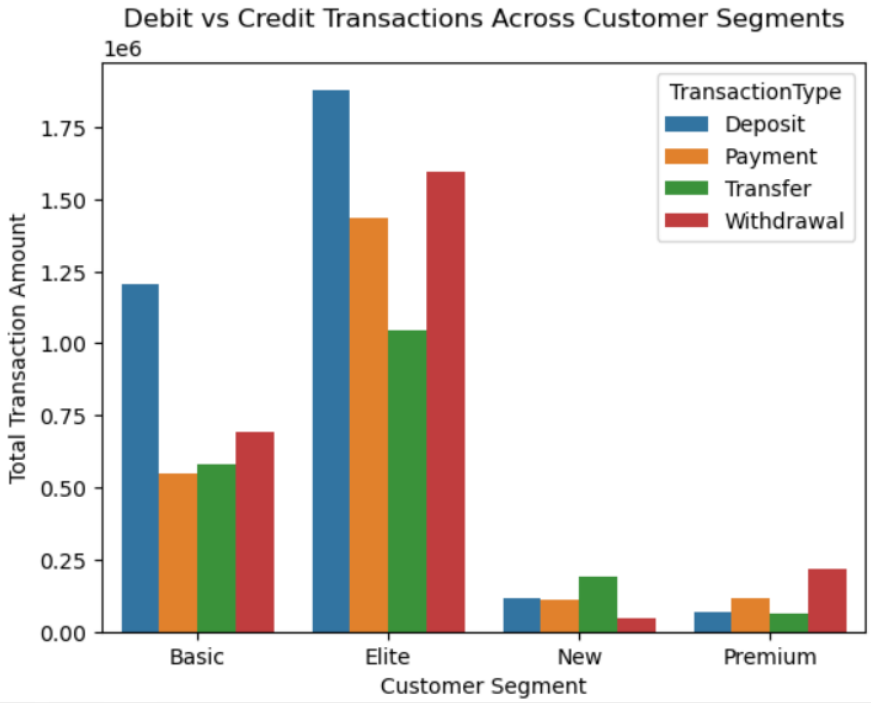
```
Customer_Interaction = br.groupby(['AccountType', 'TransactionType'])['TransactionAmount'].sum().reset_index()
sns.barplot(x='AccountType', y='TransactionAmount', hue='TransactionType', data=Customer_Interaction)
plt.title("Customer Interaction across Account type ")
plt.xlabel("Account Type")
plt.ylabel("Transaction Amount")
plt.show()
```



current accounts show balanced transactions across types, while savings account vary. ¶

Credit accounts have higher withdrawals than transfers, and loan accounts show higher deposits with lower debits.

```
br['customer_segment'] = customer_details['segment_customers']
trend_summary= br.groupby(['customer_segment','TransactionType'])['TransactionAmount'].sum().reset_index()
sns.barplot(x='customer_segment', y='TransactionAmount', hue='TransactionType', data=trend_summary)
plt.title("Debit vs Credit Transactions Across Customer Segments")
plt.xlabel("Customer Segment")
plt.ylabel("Total Transaction Amount")
plt.show()
```



Elite customers are the most financially active and high-value users.

Basic customers have steady but moderate transaction behavior.

New customers are still onboarding and not fully engaged yet.

Premium users might be using accounts more for spending than saving.

Task 6: Hypothesis Testing

Task 6.1: Test whether high-volume transaction accounts have statistically higher average balances than low-volume accounts.

--> null hypothesis (H0): There is no significant difference in average account balance between high-volume and low-volume accounts.

--> Alternate hypothesis (H1): High-volume accounts have higher average balances than low-volume accounts.

```
high_volume_account = br[br['Total_Transactions']>5]['AccountBalance']
low_volume_account = br[br['Total_Transactions']<=4]['AccountBalance']
import scipy.stats as stats
t_stat, p_value = stats.ttest_ind(high_volume_account, low_volume_account)
print(f"T-statistic: {t_stat}, P-value: {p_value}")
```

```
T-statistic: 0.8093366851233071, P-value: 0.4186256180362353
```

--> From the above results it is indicated that The independent t-test produced a p-value of 0.4186, which is greater than the significance level of 0.05.

Therefore, we accept the null hypothesis. There is no statistically significant evidence that high-volume transaction accounts have higher average balances than low-volume accounts.

Task 6.2 Conduct hypothesis testing based on segmentation.

1. Check for the relationship between net_debit and net_credit

--> null hypothesis (H0): There is no significant differnce between the mean value of net_debit and net_credit

--> Alternative Hypothesis (H_1): There is a significant difference between the mean values of net debit and net credit transactions.we perform paired T-test

--> net_debit and net_credit belongs to same accounts

--> Observations are paired measurements

```
import scipy.stats as stats
t_stat, p_value = stats.ttest_rel(br['net_debit'],br['net_credit'])
print (f"T-statistic: {t_stat}. P-value: {p_value}")
```

```
T-statistic: -6.797032422130121. P-value: 2.0891367437288038e-11
```

-->The paired t-test shows a statistically significant difference between net debit and net credit transactions ($p < 0.05$).

--> This indicates that transaction behavior differs between debit and credit activities within the same accounts. ¶

--> The results suggest that credit transactions tend to be higher than debit transactions across the dataset.

2. To determine whether the distribution of customer status categories differs significantly from an equal distribution.

--> Null hypothesis (H0): Customer status category follows normal distribution.

--> Alternate hypothesis (H1): Customer status category does not follows normal distribution.

A Chi-Square Goodness-of-Fit test was performed to compare observed frequencies of customer status with expected frequencies.

```
counts =br['account_status'].value_counts()
observed = counts.values
expected = [sum(observed)/len(observed)]*len(observed)
from scipy import stats
chi2_stat, p_value = stats.chisquare(observed,expected)
print(f"chi-square statistic: {chi2_stat}, P-value: {p_value}")
```

```
chi-square statistic: 655.22, P-value: 1.6363302545431644e-144
```

--> We reject the null hypothesis, indicating that customer account_status categories are not evenly distributed.

--> This suggests that certain customer segments dominate the dataset, which may reflect behavioral or transactional differences among customers.

Task 7: Video Presentation

Link: https://www.loom.com/share/a950bc14af4742fc9cb4bd38d9e3c9ca